Lecture 1: Intro

1. Optimization Problem (General Form)

$$\min_x f(x) \quad \text{s.t.} \quad x \in S \subseteq \mathbb{R}^n$$

Variable: $x = (x_1, \dots, x_n)^T$

Objective function: $f : \mathbb{R}^n \to \mathbb{R}$

Feasible set: S — points satisfying all constraints

Optimal solution: $x^* = \arg\min_{x \in S} f(x)$, where $f(x^*) \le f(x)\ \forall x \in S$

Optimal value: $f(x^*)$

max ↔ min equivalence: $\max f(x) \equiv \min -f(x)$

Constraint types (Constrained NLP)

$$S = \{x \in \mathbb{R}^n \mid g_i(x) = 0,\ i = 1, \dots, m;\ h_j(x) \le 0,\ j = 1, \dots, p\}$$

2. Local vs Global Minimizers

Term	Definition
Local minimizer	$\exists \epsilon > 0$ s.t. $f(x) \ge f(x^*)\ \forall x \in S \cap B_\epsilon(x^*)$
Strict local min	$f(x) > f(x^*)\ \forall x \in S \cap B_\epsilon(x^*) \setminus \{x^*\}$
Global minimizer	$f(x) \ge f(x^*)\ \forall x \in S$
Strict global min	$f(x) > f(x^*)\ \forall x \in S \setminus \{x^*\}$

Every global minimizer is a local minimizer. The converse is **not** generally true.

3. Gradient Vector

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right)^T$$

$-\nabla f(x^*)$: direction of **steepest descent**

$\nabla f(x^*)$: direction of **steepest ascent**

Key result: $\nabla(b^T x) = b$

4. Hessian Matrix

$$H_f(x) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}\right)_{n \times n}$$

Symmetric when f has continuous second-order derivatives.

Example: $f(x) = x_1^3 + 2x_1x_2 + x_2^2$

$$\nabla f = \begin{pmatrix} 3x_1^2 + 2x_2 \\ 2x_1 + 2x_2 \end{pmatrix}, \quad H_f = \begin{pmatrix} 6x_1 & 2 \\ 2 & 2 \end{pmatrix}$$

5. Matrix Definiteness

Type	Condition
Positive semidefinite (PSD)	$x^T Ax \ge 0\ \forall x$ — all eigenvalues $\lambda \ge 0$
Positive definite (PD)	$x^T Ax > 0\ \forall x \ne 0$ — all eigenvalues $\lambda > 0$
Negative semidefinite	all eigenvalues $\lambda \le 0$
Negative definite	all eigenvalues $\lambda < 0$
Indefinite	$\exists$ both positive and negative eigenvalues

PD ⇒ PSD, but PSD ⇏ PD.

6. Optimality Conditions (Unconstrained NLP)

A point x^* is a **stationary point** if $\nabla f(x^*) = 0$.

Necessary Conditions (if x^* is a local min):

1. $\nabla f(x^*) = 0$
2. $H_f(x^*)$ is **positive semidefinite**

Sufficient Conditions (to confirm x^* is a local min):

1. $\nabla f(x^*) = 0$
2. $H_f(x^*)$ is **positive definite**

If $H_f(x^*)$ is **not** PSD ⇒ x^* is **not** a local minimizer.

7. Convex Sets & Functions

Convex set: $x, y \in D \Rightarrow \lambda x + (1-\lambda)y \in D\ \forall \lambda \in [0,1]$

Convex function:

$f(\lambda x + (1-\lambda)y) \le \lambda f(x) + (1-\lambda) f(y)\ \forall x, y \in D,\ \lambda \in [0,1]$

Strictly convex: strict inequality for distinct x, y, $\lambda \in (0,1)$

Hessian Test for Convexity (Theorem 1.25)

Hessian condition (at all $x \in D$)	Conclusion
$H_f(x)$ is PSD	f is convex
$H_f(x)$ is PD	f is **strictly** convex
$H_f(\bar{x})$ is indefinite at some $\bar{x}$	f is neither convex nor concave

Why convexity matters:

Any **local minimizer** of a convex function is a **global minimizer**.

If f is **strictly convex**, the global minimizer is **unique**.

8. Quick Reference: Calculation Steps

To find & verify a local minimizer (unconstrained):

1. Solve $\nabla f(x^*) = 0$ → find stationary points
2. Compute $H_f(x^*)$
3. Find eigenvalues of $H_f(x^*)$:
   - All > 0: local min ✓ (sufficient)
   - Any < 0: not a local min ✗ (necessary fails)

To check convexity:

1. Compute $H_f(x)$ generally
2. Check eigenvalues for all $x \in D$

To find eigenvalues of A:

1. Solve $\det(A - \lambda I) = 0$ (characteristic equation)
2. For 2×2: $\lambda^2 - \mathrm{tr}(A)\lambda + \det(A) = 0$, so $\lambda = \frac{\mathrm{tr}(A) \pm \sqrt{\mathrm{tr}(A)^2 - 4\det(A)}}{2}$
3. **Shortcuts**: $\lambda_1 + \lambda_2 = \mathrm{tr}(A)$, $\lambda_1 \lambda_2 = \det(A)$
   - PD iff $\mathrm{tr}(A) > 0$ and $\det(A) > 0$
   - Indefinite iff $\det(A) < 0$

tr(A) = sum of diagonal entries $= \lambda_1 + \lambda_2$

Determinant formulas:

2×2: $\det\begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$

3×3 (cofactor expansion along row 1):

$$\det\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = a(ei - fh) - b(di - fg) + c(dh - eg)$$

9. Basic Matrix Rules

Transpose: $(AB)^T = B^T A^T$, $(A^T)^T = A$, $(A + B)^T = A^T + B^T$

Inverse: $(AB)^{-1} = B^{-1}A^{-1}$, $(A^T)^{-1} = (A^{-1})^T$, $AA^{-1} = I$

Multiplication: $A(B + C) = AB + AC$, generally $AB \ne BA$

Determinant: $\det(AB) = \det(A)\det(B)$, $\det(A^{-1}) = \frac{1}{\det(A)}$, $\det(A^T) = \det(A)$

Rank: $\mathrm{rank}(AB) \le \min(\mathrm{rank}(A), \mathrm{rank}(B))$; A invertible iff $\mathrm{rank}(A) = n$ iff $\det(A) \ne 0$

Lecture 2: Gradient Descent & Linear Regression

1. Gradient Descent Basics

Stationary point: $\nabla f(x) = 0$ — necessary condition for a local min.

Key property of gradient (Proposition 1):

- $\nabla f(x^*)^T d$ = rate of change of f moving from x^* in direction d
- f **decreases most rapidly** along $-\nabla f(x^*)$ (steepest descent direction)
- f **increases most rapidly** along $\nabla f(x^*)$

General iterative update:

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

where $\alpha_k > 0$ is the step length/learning rate and $p^{(k)}$ is the search direction.

2. Descent Direction

$p^{(k)}$ is a **descent direction** at $x^{(k)}$ if:

$$\nabla f(x^{(k)})^T p^{(k)} < 0$$

This guarantees $\exists\, \delta > 0$ such that $f(x^{(k)} + \alpha_k p^{(k)}) < f(x^{(k)})$ for all $\alpha_k \in (0, \delta)$.

Tip: $p = -\nabla f(x^{(k)})$ always satisfies this since $\nabla f^T(-\nabla f) = -\|\nabla f\|^2 \le 0$.

3. Steepest Descent Method

Update rule:

$$x^{(k+1)} = x^{(k)} - \alpha_k \nabla f(x^{(k)})$$

Stopping criterion: $\|\nabla f(x^{(k)})\| \le \epsilon$

Step Length Options

Method	Description
Constant	Fix $\alpha_k = c$. Too small → slow; too large → diverge
Exact line search	$\alpha_k = \arg\min_{\alpha > 0} f(x^{(k)} + \alpha p^{(k)})$, solve $\phi'(\alpha) = 0$
Backtracking	Start large, shrink until Armijo condition holds

Exact Line Search

Solve: $\min_{\alpha > 0} \phi(\alpha) = f(x^{(k)} + \alpha p^{(k)})$

Set $\phi'(\alpha) = 0$ and solve for α_k.

Backtracking Line Search (Armijo)

Parameters: $\bar{\alpha} > 0$, $\rho \in (0,1)$, $c_1 \in (0,1)$ (e.g. $\bar{\alpha} = 1, \rho = 0.9, c_1 = 10^{-4}$)

Shrink $\alpha \leftarrow \rho\alpha$ until:

$$f(x^{(k)} + \alpha p^{(k)}) \le f(x^{(k)}) + c_1 \alpha \nabla f(x^{(k)})^T p^{(k)}$$

Note: RHS < LHS initially since $\nabla f^T p < 0$, so the condition requires sufficient decrease.

4. Properties of Steepest Descent (Exact Line Search)

- **Perpendicular steps:** consecutive steps are orthogonal
- **Monotone decrease:** $f(x^{(k+1)}) < f(x^{(k)})$ if $\nabla f(x^{(k)}) \ne 0$
- **Zig-zag behaviour** near the solution (inherently slow)

5. Linear Regression — One Variable

Model: $f(x) = \beta_1 x + \beta_0$

Cost function:

$$L(\beta_0, \beta_1) = \frac{1}{2}\sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)^2$$

Gradients:

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)$$

$$\frac{\partial L}{\partial \beta_1} = \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i) x_i$$

Update rule:

$$\beta_0^{(k+1)} = \beta_0^{(k)} - \alpha_k \sum_{i=1}^n (\beta_1^{(k)} x_i + \beta_0^{(k)} - y_i)$$

$$\beta_1^{(k+1)} = \beta_1^{(k)} - \alpha_k \sum_{i=1}^n (\beta_1^{(k)} x_i + \beta_0^{(k)} - y_i) x_i$$

6. Linear Regression — Multiple Variables

Model: $f(x) = \beta^T x + \beta_0, \quad \beta \in \mathbb{R}^p$

Cost function:

$$L(\beta_0, \beta) = \frac{1}{2}\sum_{i=1}^n (\beta^T x_i + \beta_0 - y_i)^2$$

Gradients:

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^n (\beta^T x_i + \beta_0 - y_i)$$

$$\frac{\partial L}{\partial \beta_j} = \sum_{i=1}^n (\beta^T x_i + \beta_0 - y_i) x_{ij} \quad j = 1, \dots, p$$

Normal Equation (Analytical Solution)

Define augmented matrix and vectors:

$$\tilde{X} = \begin{bmatrix} 1 & x_1^T \\ 1 & x_2^T \\ \vdots & \vdots \\ 1 & x_n^T \end{bmatrix} \in \mathbb{R}^{n \times (p+1)}, \quad \tilde{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$$

Normal equation:

$$\tilde{X}^T \tilde{X} \tilde{\beta} = \tilde{X}^T Y$$

Solution (if $\tilde{X}^T\tilde{X}$ invertible):

$$\tilde{\beta} = (\tilde{X}^T \tilde{X})^{-1} \tilde{X}^T Y$$

Case	Condition	Solutions
Over-determined	$n \gg p$, $\tilde{X}^T\tilde{X}$ invertible	Unique
Under-determined	$n < p$, $\tilde{X}^T\tilde{X}$ not invertible	Infinite

8. Feature Scaling (Standardisation)

$$X_{\cdot j} \leftarrow \frac{X_{\cdot j} - \mathrm{mean}(X_{\cdot j})}{\mathrm{std}(X_{\cdot j})}, \qquad Y \leftarrow \frac{Y - \mathrm{mean}(Y)}{\mathrm{std}(Y)}$$

9. Steepest Descent vs. Normal Equation

	Steepest Descent	Normal Equation
Type	Iterative	Analytical
Step length	Must be chosen	Not needed
Large p	Works well	Slow (matrix inversion)
When to use	$p > 5000$	$p \le 5000$

Lecture 3: Logistic Regression & Regularization

1. Logistic Regression

Sigmoid (Logistic) Function

$$g(z) = \frac{1}{1 + e^{-z}}, \quad 0 < g(z) < 1$$

$g(0) = 0.5$, $g(z) \to 1$ as $z \to +\infty$, $g(z) \to 0$ as $z \to -\infty$

Model

$$f(x) = g(\tilde{\beta}^T \tilde{x}), \quad \tilde{\beta} = [\beta_0; \beta], \quad \tilde{x} = [1; x]$$

Probabilistic Interpretation

$$f(x) = P(y = 1 \mid x; \tilde{\beta}), \quad 1 - f(x) = P(y = 0 \mid x; \tilde{\beta})$$

Prediction Rule

- Predict $y = 1$ if $f(x) \ge 0.5$, i.e., $\tilde{\beta}^T \tilde{x} \ge 0$
- Predict $y = 0$ if $f(x) < 0.5$, i.e., $\tilde{\beta}^T \tilde{x} < 0$

Decision Boundary

$$\beta_0 + \beta^T x = 0$$

Features p	Boundary shape
1	Point
2	Line
3	Plane
general	$(p-1)$-dimensional subspace

2. Cost Function (Negative Log-Likelihood)

Per-sample cost

$$-y_i \log(f(x_i)) - (1 - y_i)\log(1 - f(x_i)) = \begin{cases} -\log(f(x_i)) & \text{if } y_i = 1 \\ -\log(1 - f(x_i)) & \text{if } y_i = 0 \end{cases}$$

Total cost (simplified form)

$$L(\beta_0, \beta) = \sum_{i=1}^n \log(1 + e^{\beta_0 + \beta^T x_i}) - y_i(\beta_0 + \beta^T x_i)$$

3. Gradient of the Cost Function

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^n (f(x_i) - y_i)$$

$$\frac{\partial L}{\partial \beta_j} = \sum_{i=1}^n (f(x_i) - y_i) x_{ij}, \quad j = 1, \dots, p$$

Note: Same form as linear regression gradient, but $f(x_i)$ is the sigmoid output.

4. Multi-class: One-vs-Rest

For K classes, train K binary classifiers $f_1, \dots, f_K$.

Predict class $k^* = \arg\max_k f_k(x)$.

5. Regularization

Ridge (L2)

$$\text{Regularizer: } \lambda\|\beta\|^2 = \lambda \sum_{j=1}^p \beta_j^2$$

- Differentiable; shrinks all β_j toward 0 but rarely to exactly 0.

Logistic + Ridge:

$$\min \sum_{i=1}^n \log(1 + e^{\beta_0 + \beta^T x_i}) - y_i(\beta_0 + \beta^T x_i) + \lambda \sum_{j=1}^p \beta_j^2$$

Linear + Ridge (Normal Equation, assuming $\beta_0 = 0$):

$$\min_\beta \frac{1}{2}\|X\beta - Y\|^2 + \lambda\|\beta\|^2$$

$$\Rightarrow \quad \beta = (2\lambda I + X^T X)^{-1} X^T Y$$

Lasso (L1)

$$\text{Regularizer: } \lambda\|\beta\|_1 = \lambda \sum_{j=1}^p |\beta_j|$$

- Non-differentiable; forces some β_j to exactly **zero** → feature selection.
- Gradient descent **not applicable**; requires specialised solvers.

Lasso problem:

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2}\|X\beta - Y\|^2 + \lambda\|\beta\|_1$$

6. Ridge vs. Lasso Summary

	Ridge	Lasso
Penalty	$\lambda\|\beta\|^2$	$\lambda\|\beta\|_1$
Differentiable	✓	✗
Shrinks β_j	Toward 0	To exactly 0
Feature selection	✗	✓
Solver	Gradient descent / Normal eq.	Specialised (e.g. proximal)

Larger λ → stronger regularization → simpler model (risk of underfitting).

Smaller λ → weaker regularization → more complex model (risk of overfitting).

Lecture 4: Proximal Gradient Method

1. Norms

Norm	Definition
ℓ_1	$\|x\|_1 = \sum_i \|x_i\|$
ℓ_2	$\|x\|_2 = \sqrt{\sum_i x_i^2}$
ℓ_∞	$\|x\|_\infty = \max_i \|x_i\|$
ℓ_p	$\|x\|_p = (\sum_i \|x_i\|^p)^{1/p}$
Frobenius	$\|A\|_F^2 = \langle A, A\rangle = \mathrm{Tr}(A^T A)$

Inner product (matrices): $\langle A, B\rangle = \mathrm{Tr}(A^T B) = \sum_i \sum_j A_{ij} B_{ij}$

Inner product (vectors): $\langle x, y\rangle = x^T y = \sum_i x_i y_i$

2. Projection onto Closed Convex Set C

$$\Pi_C(z) = \arg\min_{x \in C} \frac{1}{2}\|x - z\|^2$$

Characterisation: $x^* = \Pi_C(z) \iff \langle z - x^*, x - x^*\rangle \le 0 \quad \forall x \in C$

Set C	$\Pi_C(z)$
$\mathbb{R}^n_+$ (positive orthant)	$\max\{z, 0\}$ (elementwise)
ℓ_2-ball $\{\|x\|_2 \le 1\}$	$z / \max\{\|z\|_2, 1\}$
S^n_+ (PSD cone), $A = Q\Lambda Q^T$	$Q \cdot \mathrm{diag}(\max\{\lambda_i, 0\}) \cdot Q^T$

3. Normal Cone

$$\mathcal{N}_C(\bar{x}) = \{z \mid \langle z, x - \bar{x}\rangle \le 0 \quad \forall x \in C\}$$

Key equivalence: $u \in \mathcal{N}_C(y) \iff y = \Pi_C(y + u)$

Property: If $\bar{x} \in \mathrm{int}(C)$, then $\mathcal{N}_C(\bar{x}) = \{0\}$

Example — $C = [0, 1]$:

$\bar{x}$	$\mathcal{N}_C(\bar{x})$
0	$(-\infty, 0]$
1	$[0, +\infty)$
$(0, 1)$	$\{0\}$
$\bar{x} \notin C$	$\emptyset$

Example — $C = \{x \in \mathbb{R}^2 : \|x\| \le 1\}$:

$\bar{x}$	$\mathcal{N}_C(\bar{x})$
$\|\bar{x}\| = 1$	$\{\lambda\bar{x} : \lambda \ge 0\}$
$\|\bar{x}\| < 1$	$\{0\}$

4. Subdifferential

$$\partial f(x) = \{v \mid f(z) \ge f(x) + \langle v, z - x\rangle \quad \forall z\}$$

- If f differentiable at x: $\partial f(x) = \{\nabla f(x)\}$
- **Global optimality:** $\bar{x}$ is a global minimiser $\iff 0 \in \partial f(\bar{x})$
- **Indicator function:** $\partial\delta_C(x) = \mathcal{N}_C(x)$ for $x \in C$

Example — $f(x) = |x|$:

x	$\partial f(x)$
$x < 0$	$\{-1\}$
$x = 0$	$[-1, 1]$
$x > 0$	$\{1\}$

Lasso sparsity condition — at optimum β:

$$\beta_i < 0 \implies [X^T(X\beta - Y)]_i = \lambda$$

$$\beta_i = 0 \iff |[X^T(X\beta - Y)]_i| \le \lambda \quad \leftarrow \text{sparsity!}$$

$$\beta_i > 0 \implies [X^T(X\beta - Y)]_i = -\lambda$$

5. Fenchel Conjugate

$$f^*(y) = \sup_x \{\langle y, x\rangle - f(x)\}$$

f^* is always convex and closed (even if f is not).

If f is closed proper convex, then $(f^*)^* = f$.

Key triple equivalence:

$$f(x) + f^*(y) = \langle x, y\rangle \iff y \in \partial f(x) \iff x \in \partial f^*(y)$$

Examples:

$f(x)$	$f^*(y)$
$\|x\|_1$	$\delta_{C'}(y)$, $C = \{y : \|y\|_\infty \le 1\}$
$\delta_C(x)$ (indicator)	$\sup\{\langle y, x\rangle : x \in C\}$ (support function)

6. Moreau Envelope & Proximal Operator

$$\mathrm{prox}_f(x) = \arg\min_y \left\{f(y) + \frac{1}{2}\|y - x\|^2\right\} \quad \leftarrow \text{proximal mapping}$$

$$M_f(x) = \min_y \left\{f(y) + \frac{1}{2}\|y - x\|^2\right\} \quad \leftarrow \text{Moreau envelope}$$

Properties:

- $\nabla M_f(x) = x - \mathrm{prox}_f(x) \quad (M_f$ is always differentiable$)$
- $\arg\min f = \arg\min M_f$
- $\mathrm{prox}_{\delta_C}(x) = \Pi_C(x)$

Moreau Decomposition:

$$x = \mathrm{prox}_f(x) + \mathrm{prox}_{f^*}(x)$$

$$\frac{1}{2}\|x\|^2 = M_f(x) + M_{f^*}(x)$$

Soft Thresholding — prox of $f(x) = \lambda|x|$:

$$\mathrm{prox}_f(x) = S_\lambda(x) = \mathrm{sign}(x) \cdot \max\{|x| - \lambda, 0\}$$

Applied elementwise to $x = [x_1; \dots; x_n]$:

$$[S_\lambda(x)]_i = \mathrm{sign}(x_i) \cdot \max\{|x_i| - \lambda, 0\}$$

Huber function (Moreau envelope of $f(x) = \lambda|x|$):

$$M_f(x) = \begin{cases} \frac{1}{2}x^2 & \text{if } |x| \le \lambda \\ \lambda|x| - \frac{\lambda^2}{2} & \text{if } |x| > \lambda \end{cases}$$

7. Proximal Gradient (PG) Method

Problem: $\min_\beta f(\beta) + g(\beta)$, where f is smooth and g is convex non-smooth.

Key insight — gradient step on f only, then prox on g:

$$\beta^{(k+1)} = \mathrm{prox}_{\alpha g}\left(\beta^{(k)} - \alpha\nabla f(\beta^{(k)})\right)$$

Algorithm:

1. Choose $\beta^{(0)}$, step size $\alpha > 0$
2. Repeat: $\beta^{(k+1)} = \mathrm{prox}_{\alpha g}\left(\beta^{(k)} - \alpha\nabla f(\beta^{(k)})\right)$
3. Until convergence

Convergence: $f(\beta^{(k)}) + g(\beta^{(k)}) - \text{optimal} \le O(1/k)$

8. Accelerated Proximal Gradient (APG) Method

Algorithm (FISTA-style): Choose $\beta^{(0)}$, step size $\alpha > 0$, $t_0 = t_1 = 1$.

Repeat:

$$\tilde{\beta}^{(k)} = \beta^{(k)} + \frac{t_k - 1}{t_{k+1}}\left(\beta^{(k)} - \beta^{(k-1)}\right) \quad \leftarrow \text{momentum}$$

$$\beta^{(k+1)} = \mathrm{prox}_{\alpha g}\left(\tilde{\beta}^{(k)} - \alpha\nabla f(\tilde{\beta}^{(k)})\right)$$

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$$

Convergence: $f(\beta^{(k)}) + g(\beta^{(k)}) - \text{optimal} \le O(1/k^2)$

Step size rule: $\alpha \in (0, 1/L)$, where L = Lipschitz constant of ∇f

9. APG Applied to Lasso

$$\min_\beta \frac{1}{2}\|X\beta - Y\|^2 + \lambda\|\beta\|_1$$

- $\nabla f(\beta) = X^T(X\beta - Y)$
- Lipschitz constant: $L = \lambda_{\max}(X^T X)$
- Step size: $\alpha = 1/L$

Iteration:

$$\tilde{\beta}^{(k)} = \beta^{(k)} + \frac{t_k - 1}{t_{k+1}}\left(\beta^{(k)} - \beta^{(k-1)}\right)$$

$$\beta^{(k+1)} = S_{\lambda/L}\left(\tilde{\beta}^{(k)} - \frac{1}{L}X^T(X\tilde{\beta}^{(k)} - Y)\right)$$

Optimality condition: β^* is optimal $\iff \beta^* = \mathrm{prox}_g(\beta^* - \nabla f(\beta^*))$

Stopping criterion (tolerance $\varepsilon > 0$):

$$\left\|\beta^{(k)} - S_\lambda\left(\tilde{\beta}^{(k)} - X^T(X\tilde{\beta}^{(k)} - Y)\right)\right\| < \varepsilon$$

10. Complexity Summary

Method	Convergence rate	Iterations to 10^{-4} error
PG	$O(1/k)$	$\sim O(10^4)$
APG	$O(1/k^2)$	$\sim O(10^2)$

- APG has the **same per-iteration cost** as PG (one prox + one gradient eval)
- **Restart trick:** rerun APG every 100–200 iterations from the latest iterate

Lecture 5: SVM

1. SVM Basics

Setup: Data (x_i, y_i) where $x_i \in \mathbb{R}^p$, $y_i \in \{-1, 1\}$, classes assumed **linearly separable**.

Classifier: $f(x) = \mathrm{sign}(\beta^T x + \beta_0)$

Distance of point x to hyperplane $H = \{\beta^T x + \beta_0 = 0\}$:

$$d = \frac{|\beta^T x + \beta_0|}{\|\beta\|}$$

Margin (distance to closest point from hyperplane):

$$\gamma = \min_{i=1,\dots,n} \frac{|\beta^T x_i + \beta_0|}{\|\beta\|}$$

2. SVM Primal Problem

$$\min_{\beta, \beta_0} \frac{1}{2}\|\beta\|^2 \quad \text{s.t.} \quad y_i(\beta^T x_i + \beta_0) \ge 1, \quad \forall i \in [n]$$

- Margin $= \frac{1}{\|\beta\|}$ at optimum
- **Support vectors**: points x_i where $y_i(\beta^T x_i + \beta_0) = 1$ (tight constraint)

3. KKT Conditions (General)

LICQ (Linear Independence Constraint Qualification): At x^*, the gradients of all **active** constraints are linearly independent:

$$\{\nabla g_i(x^*)\} \cup \{\nabla h_j(x^*) : h_j(x^*) = 0\} \text{ are linearly independent}$$

LICQ ⇒ x^* is a **regular point** ⇒ KKT conditions are **necessary** for local optimality.

For primal $\min f(x)$ s.t. $g_i(x) = 0$, $h_j(x) \le 0$, the **KKT conditions** at a regular point x^* are:

Condition	Equation
Stationarity	$\nabla f(x^*) + \sum v_i^* \nabla g_i(x^*) + \sum u_j^* \nabla h_j(x^*) = 0$
Primal feasibility	$g_i(x^*) = 0,\ h_j(x^*) \le 0$
Dual feasibility	$u_j^* \ge 0$
Complementary slackness	$u_j^* h_j(x^*) = 0$

Lagrangian: $L(x, v, u) = f(x) + \sum v_i g_i(x) + \sum u_j h_j(x)$

Dual function (always concave): $\theta(v, u) = \min_x L(x, v, u)$

Weak duality: $\theta(v, u) \le f(x)$ for any dual-feasible (v, u) and primal-feasible x

Strong duality holds under: convex f, h_j; affine g_i; and **Slater's condition** ($\exists \bar{x}$: $g_i(\bar{x}) = 0$, $h_j(\bar{x}) < 0$).

Solving the dual under $\lambda \ge 0$ (boundary search):

1. Find unconstrained optimum via $\nabla_\lambda g = 0$
2. If any $\lambda_i^* < 0$ (infeasible): optimum lies on the boundary — set $\lambda_i = 0$ and re-optimize the reduced problem
3. If the reduced optimum is still infeasible, g is decreasing on $[0, \infty)$ so $\lambda_i^* = 0$
4. Repeat for each boundary case; take the feasible solution with highest $g(\lambda)$

4. SVM Dual Problem

Lagrangian (with $\alpha \in \mathbb{R}^n_+$):

$$L(\beta, \beta_0, \alpha) = \frac{1}{2}\|\beta\|^2 + \sum_{i=1}^n \alpha_i(1 - y_i(\beta^T x_i + \beta_0))$$

Setting $\frac{\partial L}{\partial \beta} = 0$ and $\frac{\partial L}{\partial \beta_0} = 0$ gives:

$$\beta^* = \sum_{i=1}^n \alpha_i y_i x_i \qquad \sum_{i=1}^n \alpha_i y_i = 0$$

Substituting $\beta^* = \sum_i \alpha_i y_i x_i$ back into L:

$$L = \frac{1}{2}\left\|\sum_i \alpha_i y_i x_i\right\|^2 + \sum_i \alpha_i - \sum_i \alpha_i y_i \left(\sum_j \alpha_j y_j x_j\right)^T x_i - \beta_0 \underbrace{\sum_i \alpha_i y_i}_{=0}$$

$$= \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle + \sum_i \alpha_i - \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle$$

$$= \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle$$

Dual problem:

$$\max_\alpha \sum_{i=1}^n \alpha_i - \frac{1}{2}\sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0,\ \alpha_i \ge 0$$

Recovering primal from dual:

$$\beta^* = \sum_{i=1}^n \alpha_i^* y_i x_i \qquad \beta_0^* = y_k - \sum_{i=1}^n \alpha_i^* y_i \langle x_i, x_k\rangle \text{ for any } k \text{ with } \alpha_k^* > 0$$

Complementary slackness: $\alpha_i^* > 0 \Rightarrow y_i((\beta^*)^T x_i + \beta_0^*) = 1$ (i.e., x_i is a support vector)

Decision boundary (test point x):

$$f(x) = \mathrm{sign}\left(\sum_{i:\alpha_i^* > 0} \alpha_i^* y_i \langle x_i, x\rangle + \beta_0^*\right)$$

5. Kernels

Replace $\langle x_i, x_j\rangle \to K(x_i, x_j) = \langle\phi(x_i), \phi(x_j)\rangle$ in the dual.

Kernel	Formula
Linear	$K(a, b) = a^T b$
Homogeneous polynomial degree d	$K(a, b) = (a^T b)^d$
Inhomogeneous polynomial degree d	$K(a, b) = (a^T b + 1)^d$
Gaussian (RBF)	$K(a, b) = \exp\left(-\frac{\|a - b\|^2}{2\sigma^2}\right)$

Key point: Computing $K(a, b)$ is $O(p)$; computing $\phi(a)$ explicitly can be $O(p^2)$ or higher.

6. Soft-Margin SVM

When classes are **not separable**, introduce slack $\varepsilon_i \ge 0$:

$$\min_{\beta, \beta_0, \varepsilon} \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^n \varepsilon_i \quad \text{s.t.} \quad y_i(\beta^T x_i + \beta_0) \ge 1 - \varepsilon_i,\ \varepsilon_i \ge 0$$

Equivalent unconstrained form (**hinge loss**):

$$\min_{\beta, \beta_0} \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^n \max\{1 - y_i(\beta^T x_i + \beta_0), 0\}$$

Soft-margin dual:

$$\max_\alpha \sum_{i=1}^n \alpha_i - \frac{1}{2}\sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0,\ 0 \le \alpha_i \le C$$

C controls the margin–violation trade-off. Large C = hard margin.

7. SVM vs. Logistic Regression

Both use $\frac{1}{2}\|\beta\|^2$ regularization; SVM uses hinge loss $\max\{1 - z, 0\}$ (sparse support vectors), logistic uses $\log(1 + e^{-z})$ (smooth approximation to hinge, all points contribute).

Lecture 6: Block Coordinate Descent (BCD)

Target Problem

$$\min_{x \in \mathbb{R}^n} F(x) = f(x) + \sum_{i=1}^n r_i(x_i)$$

- f: convex, differentiable (smooth part)
- $r_i(x_i)$: closed proper convex, **separable** (non-smooth part)

Coordinate-wise Minimizer

Definition: $\bar{x}$ is a coordinate-wise minimizer if $\bar{x} \in \mathrm{dom} f$ and:

$$f(\bar{x} + d e_i) \ge f(\bar{x}) \quad \forall i \in [n],\ d \in \mathbb{R}$$

Equivalently, for each i:

$$\bar{x}_i \in \arg\min_{x_i} f(\bar{x}_1, \dots, \bar{x}_{i-1}, x_i, \bar{x}_{i+1}, \dots, \bar{x}_n)$$

When does coordinate-wise min ⇒ global min?

Condition	Result
f differentiable at $\bar{x}$	✓ coord-wise min ⇒ global min (since $\nabla_i f(\bar{x}) = 0\ \forall i \Rightarrow \nabla f(\bar{x}) = 0$)
f non-differentiable, non-separable	✗ not guaranteed
f non-differentiable, **separable** $r_i(x_i)$	✓ coord-wise min ⇒ global min

Coordinate Descent Algorithm

Cycle through $i = 1, 2, \dots, n$ and solve one 1D subproblem at a time:

$$x_i^{(k+1)} \leftarrow \arg\min_{x_i} f(x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}, x_i, x_{i+1}^{(k)}, \dots, x_n^{(k)}) + r_i(x_i)$$

Use updated values $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ immediately. **Cannot be parallelised.**

Example — $f(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2$, start at $(x_1, x_2) = (2, 2)$:

- Update x_1:

$$\min_{x_1} x_1^2 + 2x_1 \Rightarrow \frac{\partial}{\partial x_1} = 2x_1 + x_2 = 0 \Rightarrow x_1^{(1)} = -x_2/2 = -1$$

- Update x_2:

$$\min_{x_2} (-1)(-1) + (-1)x_2 + x_2^2 \Rightarrow \frac{\partial}{\partial x_2} = x_1 + 2x_2 = 0 \Rightarrow x_2^{(1)} = 1/2$$

After 1 step: $(x_1, x_2) = (-1, 0.5)$, moving toward $(0, 0)$.

Key Tools for Derivations

Optimality condition (convex f): $x^* \in \arg\min f(x) \iff 0 \in \partial f(x^*)$

Proximal characterisation: $y = P_g(x) \iff x \in y + \partial g(y)$

Special cases: $P_{\lambda|\cdot|}(x) = S_\lambda(x)$, $\quad P_{\delta_C}(x) = \Pi_C(x)$, $\quad \partial|0| = [-1, 1]$

Applications 1–3: Regression Variants

Let $\tilde{\beta}_i = \frac{X_{\cdot i}^T(Y - X_{-i}\beta_{-i})}{\|X_{\cdot i}\|^2}$ (OLS update for coordinate i). All three share this inner quantity:

Problem	Update rule
Linear regression	$\beta_i \leftarrow \tilde{\beta}_i$
Lasso ($+\lambda\|\beta\|_1$)	$\beta_i \leftarrow S_{\lambda/\|X_{\cdot i}\|^2}(\tilde{\beta}_i)$
Box-constrained ($l \le \beta \le u$)	$\beta_i \leftarrow \Pi_{[l_i,u_i]}(\beta_i)$

$X_{-i} = X$ with i-th column removed; $\Pi_{[l,u]}(t) = \min(u, \max(l, t))$. No step size needed.

Application 4: SVM — SMO

BCD in **blocks of 2**. Select pair (α_i, α_j) violating KKT, then solve:

$$\min_{\alpha_1,\alpha_2} \frac{1}{2}K_{11}\alpha_1^2 + \frac{1}{2}K_{22}\alpha_2^2 + y_1 y_2 K_{12}\alpha_1\alpha_2 - \alpha_1 - \alpha_2 \quad \text{s.t.} \quad y_1\alpha_1 + y_2\alpha_2 = \zeta, \quad 0 \le \alpha_1 \le C, \quad 0 \le \alpha_2 \le C$$

Reduces to a 1D quadratic over an interval → closed-form solution. Pairs are required to maintain $\sum_i \alpha_i y_i = 0$.

KKT (complementary slackness) conditions:

α_i	Margin condition
$\alpha_i = 0$	$y_i(\beta^T x_i + \beta_0) \ge 1$
$0 < \alpha_i < C$	$y_i(\beta^T x_i + \beta_0) = 1$
$\alpha_i = C$	$y_i(\beta^T x_i + \beta_0) \le 1$

Application 5: Parallel Projections

Find $x \in C_1 \cap C_2 \cap \cdots \cap C_m$ via:

$$\min_{x, y_i} \frac{1}{2}\sum_{i=1}^{m} \|y_i - x\|^2 \quad \text{s.t. } y_i \in C_i$$

BCD updates (repeats until convergence):

$$y_i \leftarrow \Pi_{C_i}(x) \quad \text{for } i = 1, \ldots, m \qquad \text{(parallelisable)}$$

$$x \leftarrow \frac{1}{m}\sum_{i=1}^{m} y_i$$

Summary: CD vs Gradient Descent

	Coordinate Descent	Gradient Descent
Step size	Not needed	Required (α)
Per-iteration cost	1D subproblem	Full gradient ∇f
Parallelisable	❌ No	✅ Yes
Typical convergence	Fewer iterations	More iterations (fixed α)

Lecture 7: NMF

Core Problem

$$\min_{W,H} \frac{1}{2}\|V - WH\|_F^2, \quad \text{s.t.} \quad W \ge 0,\ H \ge 0$$

Matrix	Dimensions	Role
V	m × n	Data (columns = data points)
W	m × r	Basis vectors
H	r × n	Coordinates/coefficients

Bi-convex (non-convex jointly, but convex in W or H separately).

Variants

Variant	Extra Constraint	Use Case
Standard NMF	—	General
Orthogonal NMF	$HH^T = I_r$	Hard clustering
Symmetric NMF	$H = W^T$, so $V \approx WW^T$	Community detection
Sparse NMF	$+\lambda_W\|W\|_1 + \lambda_H\|H\|_1$	Localized features

Key lemma (Orthogonal NMF): $H \ge 0$ and $HH^T = I_r \Rightarrow$ each column of H has **at most one positive entry**.

Algorithms

BCD / HALS
Update columns of W, then rows of H cyclically:

$$W_{\cdot i} \leftarrow \Pi_+\left(\frac{(V - W_{\cdot(-i)}H_{(-i)\cdot})H_{i\cdot}^T}{\|H_{i\cdot}\|^2}\right), \qquad H_{i\cdot} \leftarrow \Pi_+\left(\frac{W_{\cdot i}^T(V - W_{\cdot(-i)}H_{(-i)\cdot})}{\|W_{\cdot i}\|^2}\right)$$

Proximal Gradient (PG)

$$W^{(k+1)} = \Pi_+\left(W^{(k)} + \alpha_k R^{(k)}(H^{(k)})^T\right), \qquad H^{(k+1)} = \Pi_+\left(H^{(k)} + \alpha_k (W^{(k+1)T}R^{(k)}\right)$$

where $R^{(k)} = V - W^{(k)}H^{(k)}$.

Multiplicative Update (MU)

$$W \leftarrow W \odot \frac{VH^T}{WHH^T}, \qquad H \leftarrow H \odot \frac{W^TV}{W^TWH}$$

ALS (default in sklearn)

Solve unconstrained least squares for H, then W; project onto $\mathbb{R}^+$ after each step.

Key Equations

Item	Formula
Frobenius norm	$\|A\|_F^2 = \mathrm{Tr}(A^TA) = \sum_{i,j} A_{ij}^2$
ℓ_1 norm	$\|A\|_1 = \sum_{i,j}\|A_{ij}\|$
Gradient w.r.t. W	$\nabla_W f = -(V - WH)H^T$
Gradient w.r.t. H	$\nabla_H f = -W^T(V - WH)$
Projection	$\Pi_+(x) = \max(x, 0)$ element-wise
Rank inequality	$\mathrm{rank}(V) \le \mathrm{rank}_+(V) \le \min(m, n)$

Cluster Membership Rules

Orthogonal / Standard NMF: point $j \in$ cluster k if $H_{kj} > H_{ij}\ \forall i \ne k$

Symmetric NMF: node $j \in$ community k if $W_{jk} > W_{ji}\ \forall i \ne k$

Lecture 8: Recommendation Systems & Matrix Completion

1. Setup

- **Utility matrix** $R \in \mathbb{R}^{m\times n}$: R_{ij} = rating of user i on item j
- Ω: index set of known entries
- **Goal**: predict missing entries

2. Collaborative Filtering

Similarity measures (user x vs user y):

Measure	Formula	Range
Jaccard	$\frac{\|S_x \cap S_y\|}{\|S_x \cup S_y\|}$	$[0, 1]$
Cosine	$\frac{r_x^T r_y}{\|r_x\|\|r_y\|}$	$[0, 1]$
Normalized cosine	$\frac{(r_x - \bar{r}_x)^T(r_y - \bar{r}_y)}{\|r_x - \bar{r}_x\|\|r_y - \bar{r}_y\|}$	$[-1, 1]$

Prediction for user x, item i (let $N(x, i)$ = neighbors who rated i):

$$r_{xi} = \frac{\sum_{y\in N(x,i)} \mathrm{Sim}(x, y) \cdot r_{yi}}{\sum_{y\in N(x,i)} \mathrm{Sim}(x, y)}$$

Item-Item CF: same formula but swap user/item roles — find similar items $j \in N(i, x)$.

3. Latent Factor Model

Assume low-rank structure $R \approx WH$, where $W \in \mathbb{R}^{m\times k}$, $H \in \mathbb{R}^{k\times n}$, $k \ll m, n$.

$$\min_{W,H} \frac{1}{2}\sum_{(i,j)\in\Omega}(R_{ij} - W_{i\cdot}H_{\cdot j})^2 + \frac{\lambda}{2}\left(\|W\|_F^2 + \|H\|_F^2\right)$$

Gradients:

$$[\nabla W]_{it} = -\sum_{j:(i,j)\in\Omega}(R_{ij} - W_{i\cdot}H_{\cdot j})H_{tj} + \lambda W_{it}$$

$$[\nabla_H F]_{tj} = -\sum_{i:(i,j)\in\Omega}(R_{ij} - W_{i\cdot}H_{\cdot j})W_{it} + \lambda H_{tj}$$

> Non-convex in (W, H) jointly; use gradient descent or SGD (sample one rating per step).

4. Matrix Completion & Nuclear Norm

Rank minimization (NP-hard) → relax to **nuclear norm** (convex envelope of rank):

$$\min_X \mathrm{rank}(X) \xrightarrow{\text{relax}} \min_X \|X\|_* \quad \text{s.t. } X_{ij} = M_{ij},\ (i, j) \in \Omega$$

$$\|X\|_* = \sum_i \sigma_i(X) \quad \text{(sum of singular values, analogous to } \ell_1 \text{ on singular values}$$

SVD: $X = U\Sigma V^T = \sum_i \sigma_i u_i v_i^T$

Recovery guarantee: if $\mathrm{rank}(X^*) = k$ and $|\Omega| \ge C \cdot n \cdot k \cdot \log^2(n)$ (uniform random), nuclear norm minimization recovers X^* w.h.p.

5. Algorithms

Algorithm 1: SDP

$$\|X\|_* = \min_{W_1,W_2} \frac{1}{2}(\mathrm{Tr}(W_1) + \mathrm{Tr}(W_2)) \quad \text{s.t.} \quad \begin{bmatrix} W_1 & X \\ X^T & W_2 \end{bmatrix} \succeq 0$$

Plug in $X_{ij} = M_{ij}$ constraints → standard SDP, solve with CVX/Mosek.

Algorithm 2: Proximal Gradient

$$\min_X \underbrace{\frac{1}{2}\sum_{(i,j)\in\Omega}(X_{ij} - M_{ij})^2}_{f(X)} + \underbrace{\mu\|X\|_*}_{g(X)}$$

Proximal map of $\mu\|\cdot\|_*$ — soft-threshold singular values:

1. SVD: $Y = U\,\mathrm{diag}(\sigma)\,V^T$
2. $\gamma_i = \max(\sigma_i - \mu, 0)$
3. $\mathrm{prox}_{\mu\|\cdot\|_*}(Y) = U\,\mathrm{diag}(\gamma)\,V^T$

PG iteration: $X^{(t+1)} \leftarrow \mathrm{prox}_{\mu\|\cdot\|_*}\left(X^{(t)} - \alpha\nabla f(X^{(t)})\right)$

6. Method Comparison

Method	Pros	Cons
User/Item CF	Simple, interpretable	Doesn't scale; cold-start problem
Latent Factor	Scalable	Non-convex, needs tuning
Nuclear Norm (SDP/PG)	Convex, theoretical guarantees	Expensive (SVD per iteration)

Lecture 9: PCA, Robust PCA & ADMM

1. PCA

Goal: Find direction w with $\|w\| = 1$ of greatest variance in data $X \in \mathbb{R}^{n\times p}$.

Covariance matrix:

$$\Sigma = \frac{1}{n}\sum_{i=1}^{n} x_i x_i^T = \frac{1}{n}X^TX \quad \text{(assuming zero-mean data)}$$

Optimization problem:

$$\max_w w^TX^TXw \quad \text{s.t.} \quad \|w\| = 1$$

Equivalent to maximizing the **Rayleigh quotient**:

$$\max_w \frac{w^TX^TXw}{w^Tw}$$

Solution: w = eigenvector of X^TX with the **largest eigenvalue** λ_1.

Steps to find the principal subspace:

1. Compute $\Sigma = \frac{1}{n}\sum_{i=1}^{n} x_i x_i^T$
2. Find eigenvalues via $\det(\Sigma - \lambda I) = 0$
3. Find eigenvector w for the largest λ via $(\Sigma - \lambda I)w = 0$

k**-th component:** eigenvector of X^TX with the k-th largest eigenvalue.

Equivalent low-rank formulation:

$$\min_{L:\,\mathrm{rank}(L)=r} \|X - L\|_F$$

> ⚠️ PCA is **sensitive to outliers** — a few corrupted entries can distort the principal components.

2. Robust PCA

Model: $X = L_0 + S_0$, where L_0 is low-rank and S_0 is sparse.

Non-convex (NP-hard) formulation:

$$\min_{L,S} \mathrm{rank}(L) + \lambda\|S\|_0 \quad \text{s.t.} \quad L + S = X$$

Convex relaxation:

$$\min_{L,S} \|L\|_* + \lambda\|S\|_1 \quad \text{s.t.} \quad L + S = X$$

Term	Meaning	Relaxes
$\|L\|_* = \mathrm{tr}(\sqrt{L^TL})$	Nuclear norm = sum of singular values	$\mathrm{rank}(L)$
$\|S\|_1$	Sum of absolute values of all entries	$\|S\|_0$

Recommended λ:

$$\lambda = \frac{1}{\sqrt{\max(m, n)}}$$

3. ADMM

Target problem (2-block separable):

$$\min_{y,z} f(y) + g(z) \quad \text{s.t.} \quad Ay + Bz = c$$

Augmented Lagrangian ($\sigma > 0$):

$$\mathcal{L}\sigma(y, z, x) = f(y) + g(z) + \langle x, Ay + Bz - c\rangle + \frac{\sigma}{2}\|Ay + Bz - c\|^2$$

Using completing the square, equivalently:

$$\mathcal{L}_\sigma(y, z, x) = f(y) + g(z) + \frac{\sigma}{2}\left\|Ay + Bz - c + \sigma^{-1}x\right\|^2 - \frac{1}{2\sigma}\|x\|^2$$

ADMM iterations:

$$y^{(k+1)} \leftarrow \arg\min_y \mathcal{L}_\sigma\left(y, z^{(k)}, x^{(k)}\right)$$

$$z^{(k+1)} \leftarrow \arg\min_z \mathcal{L}_\sigma\left(y^{(k+1)}, z, x^{(k)}\right)$$

$$x^{(k+1)} \leftarrow x^{(k)} + \tau\sigma\left(Ay^{(k+1)} + Bz^{(k+1)} - c\right)$$

Parameters: $\sigma > 0, \quad 0 < \tau < \frac{1 + \sqrt{5}}{2} \approx 1.618$

Key completing-the-square identity:

$$\langle X, Y\rangle + \frac{\sigma}{2}\|Y\|^2 = \frac{\sigma}{2}\left\|Y + \sigma^{-1}X\right\|^2 - \frac{1}{2\sigma}\|X\|^2$$

Proximal operator:

$$\mathbf{P}_{\frac{1}{\sigma}f}(v) = \arg\min_y \left\{\frac{1}{\sigma}f(y) + \frac{1}{2}\|y - v\|^2\right\}$$

4. ADMM for Robust PCA

Problem:

$$\min_{L,S} \|L\|_* + \lambda\|S\|_1 \ \text{s.t.}\ L + S = X \quad \Rightarrow \quad A = I,\ B = I,\ c = X$$

Augmented Lagrangian:

$$\mathcal{L}_\sigma(L, S, Z) = \|L\|_* + \lambda\|S\|_1 - \frac{1}{2}\left\|L + S - X + \sigma^{-1}Z\right\|_F^2 - \frac{1}{2\sigma}\|Z\|_F^2$$

Subproblem-L (singular value soft-thresholding):

1. Compute SVD:
$$T^{(k)} = X - S^{(k)} - \sigma^{-1}Z^{(k)} = U^{(k)}\,\mathrm{Diag}(d^{(k)})\,(V^{(k)})^T$$
2. Shrink singular values: $\gamma^{(k)} = S_{1/\sigma}(d^{(k)})$
3. Update: $L^{(k+1)} = U^{(k)}\,\mathrm{Diag}(\gamma^{(k)})\,(V^{(k)})^T$

Subproblem-S (element-wise soft-thresholding):

$$S^{(k+1)} = S_{\lambda/\sigma}\left(X - L^{(k+1)} - \sigma^{-1}Z^{(k)}\right)$$

Multiplier update:

$$Z^{(k+1)} = Z^{(k)} + \tau\sigma\left(L^{(k+1)} + S^{(k+1)} - X\right)$$

5. ADMM for Lasso

Lasso: $\min_{\beta\in\mathbb{R}^p} \frac{1}{2}\|X\beta - Y\|^2 + \lambda\|\beta\|_1, \quad X \in \mathbb{R}^{n\times p},\ Y \in \mathbb{R}^n$

Primal ADMM — use when $p < n$

Introduce slack $u = \beta$:

$$\min_{\beta,u} \frac{1}{2}\|X\beta - Y\|^2 + \lambda\|u\|_1 \quad \text{s.t.} \quad \beta - u = 0$$

Iterations:

$$\beta^{(k+1)} = \left(\sigma I + X^TX\right)^{-1}\left(X^TY + \sigma u^{(k)} - \xi^{(k)}\right) \quad [p \times p \text{ linear system}]$$

$$u^{(k+1)} = S_{\lambda/\sigma}\left(\beta^{(k+1)} + \sigma^{-1}\xi^{(k)}\right)$$

$$\xi^{(k+1)} = \xi^{(k)} + \tau\sigma\left(\beta^{(k+1)} - u^{(k+1)}\right)$$

Dual ADMM — use when $n < p$

Dual problem:

$$\min_{y,v} \frac{1}{2}\|y\|^2 - \langle y, Y\rangle + \delta_{B_\lambda}(v) \quad \text{s.t.} \quad X^Ty + v = 0$$

Iterations:

$$y^{(k+1)} = \left(I + \sigma XX^T\right)^{-1}\left(Y - X\left(\beta^{(k)} + \sigma v^{(k)}\right)\right) \quad [n \times n \text{ linear system}]$$

$$v^{(k+1)} = \Pi_{B_\lambda}\left(-X^Ty^{(k+1)} - \sigma^{-1}\beta^{(k)}\right)$$

$$\beta^{(k+1)} = \beta^{(k)} + \tau\sigma\left(X^Ty^{(k+1)} + v^{(k+1)}\right)$$

where $B_\lambda = \{v \mid \|v\|_\infty \le \lambda\}$ and $\Pi_{B_\lambda}(s)_i = \mathrm{clamp}(s_i, -\lambda, \lambda)$.

6. Practical Notes

- σ **too large** ⇒ insufficient minimization of $f + g$
- σ **too small** ⇒ insufficient feasibility enforcement
- Common step lengths: $\tau = 1$ or $\tau = 1.618 \approx \frac{1+\sqrt{5}}{2}$
- Pre-compute and cache $(\sigma I + X^TX)^{-1}$ or $(I + \sigma XX^T)^{-1}$ before iterating
- Choose **primal vs. dual ADMM** based on problem dimension:
  - $p < n$ ⇒ primal ADMM ($p \times p$ system)
  - $n < p$ ⇒ dual ADMM ($n \times n$ system)

Lecture 10: Gaussian Graphical Models

1. Conditional Independence

Definition: $X \perp\!\!\!\perp Y \mid Z$ if $p(x, y \mid z) = p(x \mid z)\,p(y \mid z)$

Factorization: If $X \perp\!\!\!\perp Y \mid Z$, then $p(x, y, z) = g(x, z)\,h(y, z)$

2. Undirected Graph

- Graph $G = (V, E)$, vertex set $V = \{1, \ldots, p\}$, edge set $E \subset V \times V$
- Undirected: $(s, t) \in E \iff (t, s) \in E$; no self-loops

Markov Property: x is Markov w.r.t. G if for every cut set S separating A and B:

$$x_A \perp\!\!\!\perp x_B \mid x_S$$

3. Multivariate Gaussian

$$f(x) = (2\pi)^{-p/2}\det(\Theta)^{1/2}\exp\left(-\frac{1}{2}x^T\Theta x\right), \quad x \sim \mathcal{N}(0, \Sigma)$$

- **Precision matrix:** $\Theta = \Sigma^{-1} \in \mathbb{S}_{++}^p$
- $\det(\Sigma) = \prod_j \lambda_j(\Sigma), \quad \det(\Theta) = 1/\det(\Sigma)$

4. Gaussian Graphical Model (GGM)

Key fact: For $x \sim \mathcal{N}(0, \Sigma)$:

$$x_s \perp\!\!\!\perp x_t \mid x_{V\setminus\{s,t\}} \iff \Theta_{st} = 0$$

Edge set: $E = \{(s, t) \mid \Theta_{st} \ne 0,\ s < t\}$

> Zero off-diagonal entry in $\Theta \longleftrightarrow$ no edge $\longleftrightarrow$ conditional independence

Steps to draw the graph:

1. Compute $\Theta = \Sigma^{-1}$
2. Find all (s, t) with $\Theta_{st} \ne 0$, $s < t$ → these are the edges
3. Plot

5. Maximum Likelihood Estimation (MLE)

Given n i.i.d. samples, the **sample covariance matrix** is:

$$S = \frac{1}{n}\sum_{i=1}^{n} x^{(i)}(x^{(i)})^T$$

MLE problem:

$$\min_{\Theta\in\mathbb{S}_{++}^p} -\log\det(\Theta) + \langle S, \Theta\rangle$$

- MLE solution (when it exists): $\hat{\Theta} = S^{-1}$
- Solution **fails** when $n < p$ (S is rank-deficient)

Log-det function: $h(\Theta) = -\log\det(\Theta)$

- Convex on $\mathbb{S}_{++}^p$
- Gradient: $\nabla h(\Theta) = -\Theta^{-1}$

6. Graphical Lasso

Adds ℓ_1 penalty on off-diagonal entries to encourage sparsity:

$$\min_{\Theta\in\mathbb{S}_{++}^p} -\log\det(\Theta) + \langle S, \Theta\rangle + \lambda\|\Theta\|_{1,\text{off}}$$

where $\|\Theta\|_{1,\text{off}} = \sum_{s\ne t}|\Theta_{st}|$

7. ADMM for Graphical Lasso

Reformulate with splitting variable Y, Lagrange multiplier Z:

Step	Update
Θ-update	$T = Y - \sigma^{-1}(S + Z)$; eigen-decomp $T = Q\,\mathrm{Diag}(\rho)\,Q^T$; then $\gamma_j = \frac{1}{2}\left(\rho_j + \sqrt{\rho_j^2 + 4/\sigma}\right)$; $\Theta \leftarrow Q\,\mathrm{Diag}(\gamma)\,Q^T$
Y-update	$Y \leftarrow S_{\lambda/\sigma}^{\text{off}}(\Theta + \sigma^{-1}Z)$ — soft-threshold **off-diagonal** only
Z-update	$Z \leftarrow Z + \tau\sigma(\Theta - Y)$, with $0 < \tau < \frac{1+\sqrt{5}}{2}$

Soft-threshold operator: $S_\alpha(x) = \mathrm{sign}(x)\max(|x| - \alpha, 0)$

Proximal map of log-det:

$$P_{\frac{1}{\sigma}h}(Y) = Q\,\mathrm{Diag}(\gamma)\,Q^T, \quad \gamma_j = \frac{1}{2}\left(\rho_j + \sqrt{\rho_j^2 + 4/\sigma}\right)$$

8. Neighbourhood Selection

Idea: Regress each variable on all others using Lasso.

For vertex j, solve:

$$\beta^j = \arg\min_{\beta\in\mathbb{R}^{p-1}} \frac{1}{2}\|X_{\cdot,-j}\beta - X_{\cdot j}\|^2 + \lambda\|\beta\|_1$$

Neighbourhood: $\mathcal{N}(j) = \{t \mid \beta_t^j \ne 0\}$

Graph construction:

- **AND rule:** edge (s, t) iff $s \in \mathcal{N}(t)$ **and** $t \in \mathcal{N}(s)$
- **OR rule:** edge (s, t) iff $s \in \mathcal{N}(t)$ **or** $t \in \mathcal{N}(s)$

Lecture 11: Second Order Methods

1. Rates of Convergence

Type	Condition	Example
Q-linear	$\frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|} \le r \in (0, 1)$	$1 + 0.8^k$
Q-superlinear	$\lim_{k\to\infty} \frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|} = 0$	$1 + k^{-k}$
Q-quadratic	$\frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|^2} \le M$ for some $M > 0$	$1 + (0.8)^{2^k}$

Hierarchy: Q-quadratic > Q-superlinear > Q-linear (eventually faster)

Key facts:

- Q-quadratic ⇒ Q-superlinear ⇒ Q-linear
- $1/k$ is **not** Q-linearly convergent; $1/k!$ is Q-superlinear but **not** Q-quadratic

2. General Framework

For $\min_x f(x)$, differentiable:

$$x^{(k+1)} = x^{(k)} - \alpha_k P_k \nabla f(x^{(k)})$$

P_k	Method
I	Gradient (steepest) descent
$[H_f(x^{(k)})]^{-1}$	Pure Newton's method
$[H_f(x^{(k)}) + \tau_k I]^{-1}$	Modified Newton
$H_k \approx [H_f(x^{(k)})]^{-1}$	Quasi-Newton (BFGS)

3. Pure Newton's Method

Derivation: Minimize 2nd-order Taylor approximation of f.

Newton direction:

$$p^{(k)} = -[H_f(x^{(k)})]^{-1}\nabla f(x^{(k)})$$

equivalently, solve the **linear system**:

$$H_f(x^{(k)})\,p = -\nabla f(x^{(k)})$$

Update: $x^{(k+1)} = x^{(k)} + p^{(k)}$ (step size = 1)

Descent direction condition: $p^{(k)}$ is a descent direction iff $H_f(x^{(k)}) \succ 0$:

$$\nabla f(x^{(k)})^T p^{(k)} = -(p^{(k)})^T H_f(x^{(k)}) p^{(k)} < 0$$

Convergence: Q-quadratic (locally, near x^*), given:

1. f twice differentiable
2. $H_f(\cdot)$ locally Lipschitz at x^*: $\|H_f(x^*) - H_f(x)\| \le L\|x^* - x\|$
3. Sufficient conditions (SOSC) at x^*

Limitations: May diverge from remote starting points; requires Hessian ($O(n^3)$ to solve).

4. Practical Newton's Method

4a. Line Search Newton with Modification

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}, \quad p^{(k)} = -[H_f(x^{(k)}) + \tau_k I]^{-1}\nabla f(x^{(k)})$$

Hessian modification: If $H_f(x^{(k)}) \not\succ 0$, choose $\tau_k > -\lambda_{\min}(H_f(x^{(k)}))$ so $B_k = H_f(x^{(k)}) + \tau_k I \succ 0$.

- If τ_k too large: $p^{(k)} \approx -\tau_k^{-1}\nabla f(x^{(k)})$ (degenerates to slow gradient descent)

Line search (Armijo backtracking):

$$f(x^{(k)} + \alpha p^{(k)}) \le f(x^{(k)}) + c_1\alpha\nabla f(x^{(k)})^T p^{(k)}$$

Start with $\alpha = 1$, reduce by $\alpha \leftarrow \rho\alpha$ until satisfied.

Convergence rate:

- If $H_f(x^*) \succ 0$: $\tau_k \to 0$, reduces to pure Newton → **Q-quadratic**
- Otherwise: may be only **linear**

4b. BFGS (Quasi-Newton)

Approximates $H_k \approx [H_f(x^{(k)})]^{-1}$ without computing the true Hessian.

Secant equation (the key constraint on $B_{k+1} \approx H_f$):

$$B_{k+1}s_k = y_k$$

where $s_k = x^{(k+1)} - x^{(k)}, \quad y_k = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)})$

Curvature condition (required for $B_{k+1} \succ 0$): $y_k^T s_k > 0$

BFGS update formula (updates inverse Hessian approximation H_k):

$$H_{k+1} = (I - \rho_k s_k y_k^T)H_k(I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \rho_k = \frac{1}{y_k^T s_k}$$

Equivalent B_k update (rank-2):

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

Per-iteration cost: $O(n^2)$. **Convergence:** Q-superlinear.

5. Trust-Region Method

Model function:

$$m_k(p) = f(x^{(k)}) + \nabla f(x^{(k)})^T p + \frac{1}{2}p^T B_k p$$

Subproblem: $\min_p\ m_k(p) \quad \text{s.t.} \quad \|p\| \le \Delta_k$

Ratio for adapting radius:

$$\rho_k = \frac{f(x^{(k)}) - f(x^{(k)} + p^{(k)})}{m_k(0) - m_k(p^{(k)})} = \frac{\text{actual reduction}}{\text{predicted reduction}}$$

ρ_k	Action
$< 1/4$	Shrink: $\Delta_{k+1} = \frac{1}{4}\Delta_k$; reject step
$> 3/4$ and $\|p^{(k)}\| = \Delta_k$	Expand: $\Delta_{k+1} = \min(2\Delta_k, \hat{\Delta})$
otherwise	Keep: $\Delta_{k+1} = \Delta_k$
$> \eta$	Accept step: $x^{(k+1)} = x^{(k)} + p^{(k)}$

Full step (when $B_k \succ 0$ and $\|B_k^{-1}\nabla f(x^{(k)})\| \le \Delta_k$):

$$p^{(k)} = -B_k^{-1}\nabla f(x^{(k)})$$

6. Proximal Newton (Non-smooth g)

For $\min_x f(x) + g(x)$ (f smooth, g non-smooth):

$$x^{(k+1)} = \arg\min_x \left\{\nabla f(x^{(k)})^T(x - x^{(k)}) + \frac{1}{2}(x - x^{(k)})^T B_k(x - x^{(k)}) + g(x\right.$$

Quick Reference: Convergence Rates

Method	Rate
Gradient descent	Q-linear
Newton's (pure, local)	Q-quadratic
Newton's (modified + line search, at x^* with $H \succ 0$)	Q-quadratic
Newton's (at x^* with $H \not\succ 0$)	Q-linear
BFGS	Q-superlinear
Trust-region Newton (at SOSC point)	Q-superlinear

Lecture 12: Clustering

1. Core Idea

Clustering groups data so that:

- **Intra-cluster distance** is **small** (similar points together)
- **Inter-cluster distance** is **large** (different groups far apart)

2. Distance Between Two Objects

$$D(a, b) = \|a - b\|_2 = \sqrt{\sum_{i=1}^{p}(a_i - b_i)^2}$$

Other options: $\|a - b\|_1$, $\|a - b\|_\infty$, cosine similarity.

3. Distance Between Two Clusters

Linkage	Formula	Description
Single	$\min_{a\in C_1, b\in C_2} D(a, b)$	Shortest pairwise distance
Complete	$\max_{a\in C_1, b\in C_2} D(a, b)$	Longest pairwise distance
Average ⭐	$\frac{1}{\|C_1\|\|C_2\|}\sum_{a\in C_1, b\in C_2} D(a, b)$	Average of all pairwise distances

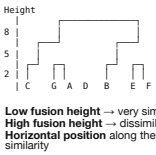
> ⭐ Average linkage is the most widely used.

4. Hierarchical Clustering (Bottom-Up / Agglomerative)

Algorithm:

1. Start: each object is its own cluster → n clusters
2. Find the pair of clusters with **smallest** $L(C_i, C_j)$
3. Merge them into one cluster
4. Repeat until 1 cluster remains

Output: A **dendrogram** (tree diagram)



- **Low fusion height** → very similar objects
- **High fusion height** → dissimilar objects
- **Horizontal position** along the x-axis means **nothing** about similarity

Reading the Dendrogram

Q	How to answer
Are X and Y similar?	Check how low they fuse — lower = more similar
How different are two clusters?	Read the **height** on the vertical axis
Is A more similar to B than to C?	**Cannot** determine from dendrogram alone

Identifying Clusters: Horizontal Cut

Make a horizontal cut at height $y = a$:

- Number of vertical lines crossed = **number of clusters**

Choosing the best cut: pick the cut with the **largest vertical range** (most room to move up/down without hitting a horizontal branch).

```
Cut at y=6  →  3 clusters: {C,G,A,D}, {B}, {E,F}   (range 1.91) ✓
Cut at y=3.4 →  5 clusters: {C,G}, {A,D}, {B}, {E}, {F}  (range 0
```

5. K-Means Clustering

Objective: minimise total intra-cluster distance

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K}\frac{1}{|C_k|}\sum_{i,j\in C_k}\|x_i - x_j\|^2$$

Algorithm:

```
Initialise: place K centroids μ₁, ..., μ_K at random

Repeat until assignments don't change:
  (1) Assign each xᵢ to nearest centroid:
          argmin_k  D(xᵢ, μ_k)

  (2) Recompute centroid of each cluster:
          μ_k = (1/|C_k|) Σ_{i∈C_k} xᵢ
```

Choosing K (Elbow Method)

Plot the objective function vs K. Look for the **"elbow"** — the point of most abrupt decrease.

```
Objective
  |  *
  |    *
  |      *
  |        * * * * *
  +--1---2---3----4---5--- K
                 ↑
            elbow → K=2 or K=3
```

6. Hierarchical vs K-Means Summary

	Hierarchical	K-Means
K specified?	No (cut dendrogram after)	Yes, upfront
Output	Dendrogram	Flat cluster assignments
Sensitive to init?	No	Yes — run multiple times
Key choice	Linkage type + cut height	K, initialisation
Overlapping clusters?	No	No